

Software Requirements Specification (SRS)

Animal Rescue Network

1. Introduction

1.1 Purpose

The purpose of this document is to describe the requirements of the **Animal Rescue Network**, a web-based platform developed using the **MERN stack (MongoDB, Express.js, React.js, Node.js)**. The system enables citizens to report stray animals in need of rescue, allows volunteers to respond to rescue requests, and provides a donation system for medical treatment of rescued animals.

This document specifies the **functional requirements, non-functional requirements, system features, and constraints** of the system.

1.2 Scope

The **Animal Rescue Network** aims to improve the efficiency of animal rescue operations within a city by connecting citizens, volunteers, and donors on a single platform.

The system allows:

- Citizens to report stray animals needing rescue
- Volunteers to claim rescue requests
- Admins to monitor rescue activities
- Donors to contribute towards medical treatments

The system includes features such as **photo uploads, geo-tagging of rescue locations, volunteer claim tracking, and a donation portal for medical cases.**

1.3 Definitions, Acronyms, and Abbreviations

Term	Meaning
MERN	MongoDB, Express.js, React.js, Node.js
SRS	Software Requirements Specification
Admin	System administrator managing the platform
Volunteer	Registered user who performs rescue operations
Reporter	User who reports stray animals
Geo-tag	Location tagging using map coordinates

1.4 References

- IEEE 830 Software Requirements Specification Standard
 - MERN Stack Documentation
 - MongoDB Documentation
 - React Documentation
-

1.5 Overview

This document describes the **overall system structure, system features, functional requirements, non-functional requirements, and constraints** of the Animal Rescue Network.

2. Overall Description

2.1 Product Perspective

The **Animal Rescue Network** is a **web-based application** that connects users and volunteers to help rescue stray animals. The system provides an interface for reporting rescue requests, managing volunteers, tracking rescue activities, and accepting donations.

The system architecture follows the **MERN stack architecture**:

Frontend: React

Backend: Node.js + Express

Database: MongoDB

2.2 Product Functions

The main functions of the system include:

1. User Registration and Login
 2. Reporting stray animals with image and location
 3. Volunteer rescue claim system
 4. Tracking rescue status
 5. Donation portal for medical cases
 6. Admin monitoring system
 7. Notification system for rescue updates
-

2.3 User Classes and Characteristics

1. Citizen / Reporter

- Reports stray animals needing rescue
- Uploads photos and location
- Tracks rescue progress

2. Volunteer

- Views rescue requests
- Claims rescue tasks
- Updates rescue status

3. Donor

- Donates money for animal medical treatment
- Views case details before donating

4. Admin

- Manages users
 - Monitors rescue operations
 - Verifies rescue posts
 - Tracks donation records
-

2.4 Operating Environment

Component	Technology
Frontend	React.js
Backend	Node.js + Express.js
Database	MongoDB
Hosting	Vercel / Render / AWS
Browser	Chrome, Firefox, Edge
OS	Windows / Linux / macOS

2.5 Design and Implementation Constraints

- System must use **MERN stack technologies**
 - Image storage must support **photo uploads**
 - Geo-tagging requires **map integration**
 - Payment gateway must support **secure transactions**
 - Database must ensure **data integrity**
-

2.6 Assumptions and Dependencies

- Users have internet access
 - Volunteers are available in the city
 - Map services (Google Maps / OpenStreetMap) are available
 - Payment gateway services are available
-

3. System Features

3.1 User Registration and Login

Description

Users must create an account to access platform features.

Functional Requirements

FR1: The system shall allow users to register with name, email, phone number, and password.

FR2: The system shall allow users to log in using email and password.

FR3: The system shall verify user credentials before granting access.

FR4: The system shall allow users to log out securely.

FR5: The system shall allow password reset.

3.2 Report Animal Rescue Request

Description

Users can report stray animals requiring rescue assistance.

Functional Requirements

FR6: The system shall allow users to create a rescue request.

FR7: The system shall allow uploading photos of the animal.

FR8: The system shall allow geo-tagging the rescue location.

FR9: The system shall allow adding a description of the animal's condition.

FR10: The system shall store rescue requests in the database.

FR11: The system shall display rescue requests on the dashboard.

3.3 Volunteer Claim System

Description

Volunteers can claim rescue requests and respond to emergencies.

Functional Requirements

FR12: The system shall allow volunteers to view rescue requests.

FR13: The system shall allow volunteers to claim rescue tasks.

FR14: The system shall mark the rescue request as "Claimed".

FR15: The system shall allow volunteers to update rescue status.

FR16: The system shall allow volunteers to mark rescue as "Completed".

3.4 Rescue Tracking System

Description

Users can track the status of their rescue request.

Functional Requirements

FR17: The system shall allow users to view rescue request status.

FR18: The system shall display status such as:

- Pending
- Claimed
- In Progress
- Completed

FR19: The system shall allow status updates from volunteers.

3.5 Donation Portal

Description

Donors can contribute money to support medical treatment of rescued animals.

Functional Requirements

FR20: The system shall allow creation of donation campaigns for animals.

FR21: The system shall allow donors to view case details.

FR22: The system shall allow donors to make secure payments.

FR23: The system shall record donation transactions.

FR24: The system shall display total funds collected.

3.6 Admin Management

Description

Admins manage users and rescue operations.

Functional Requirements

FR25: The system shall allow admin login.

FR26: The system shall allow admin to approve rescue posts.

FR27: The system shall allow admin to remove inappropriate content.

FR28: The system shall allow admin to monitor rescue activities.

FR29: The system shall allow admin to manage donation records.

4. External Interface Requirements

4.1 User Interfaces

The system shall provide:

- User Dashboard
 - Volunteer Dashboard
 - Admin Panel
 - Rescue Request Form
 - Donation Portal
 - Map-based location display
-

4.2 Hardware Interfaces

The system requires:

- Internet-connected device
 - Smartphone or computer
 - GPS-enabled device for geo-tagging
-

4.3 Software Interfaces

The system interacts with:

- MongoDB database
- Payment gateway APIs
- Map services APIs
- Cloud image storage

4.4 Communication Interfaces

- HTTP/HTTPS communication
 - RESTful APIs for client-server communication
-

5. Non-Functional Requirements

5.1 Performance Requirements

NFR1: The system shall support **at least 500 concurrent users**.

NFR2: Rescue request submission shall complete within **3 seconds**.

NFR3: Page loading time shall not exceed **2 seconds**.

5.2 Security Requirements

NFR4: User passwords shall be encrypted.

NFR5: The system shall implement **authentication and authorization**.

NFR6: Payment transactions shall use **secure encryption protocols**.

NFR7: The system shall protect against **SQL injection and XSS attacks**.

5.3 Usability Requirements

NFR8: The system shall provide an intuitive user interface.

NFR9: The system shall support mobile-friendly design.

NFR10: The system shall provide clear navigation and instructions.

5.4 Reliability Requirements

NFR11: The system shall maintain **99% uptime**.

NFR12: The system shall ensure data consistency.

5.5 Maintainability Requirements

NFR13: The code shall follow modular architecture.

NFR14: The system shall support easy updates and bug fixes.

5.6 Scalability Requirements

NFR15: The system shall support increasing numbers of users and rescue requests.

5.7 Safety Requirements

NFR16: The system shall prevent false rescue reports through verification.

NFR17: The system shall allow admin moderation of rescue requests.

6. Business Rules

BR1: Only registered users can report rescue cases.

BR2: Only registered volunteers can claim rescue tasks.

BR3: Each rescue request can be claimed by **only one volunteer at a time**.

BR4: Donations must be linked to **verified medical cases**.

BR5: Admin approval may be required before public visibility of rescue requests.

7. Future Enhancements

- Mobile application
- Real-time notifications
- AI-based animal injury detection
- Integration with veterinary hospitals
- Emergency rescue alerts